

World War 3

Private-Server Revival

Project Progress & Concept

Updated 22 July 2026

The concept	World War 3 is an online-only shooter whose live backend is no longer usable. This project makes the user's own, legally-installed copy playable again by standing up local replacements for the game's backend services and steering the client to them — the same idea as a community private server for a retired MMO.
The goal	Drive the installed Steam client — through the real Easy Anti-Cheat bootstrap and the user's own account — all the way to a working Main Menu against local REST, WebSocket Hub and XMPP mocks, with no live game servers involved.
Scope & ownership	Everything runs on the owner's own machine, with the owner's own game licence and identity. No anti-cheat is defeated (EAC runs normally in offline mode); nothing is redistributed. This is interoperability / preservation work, and the associated Terms-of-Service and legal considerations are the owner's to weigh.

Owner: georgek / GEORGEKGAMING[STEAM]

Workspace: /media/georgek/Work/Dev_Work/GameDev/WW3

Platform: Linux (workstation "Megatron") + Proton Experimental

Contents

1. What this project is (concept & goal)
2. How the game gets online (recovered architecture)
3. The emulation strategy
4. What was built before this session
5. This session — root-cause breakthroughs
6. The DNS / c-ares discovery (why redirects failed)
7. Launch mechanics & Easy Anti-Cheat
8. Current system state
9. Remaining work to reach Main Menu
10. Glossary

1. What this project is (concept & goal)

World War 3 (Steam app 674020) is a multiplayer-only first-person shooter. Like most online games, the client is only half of the product: on startup it must reach a chain of backend services to authenticate the player, load their profile, and connect them to the lobby “Hub”. When those services are unavailable, the client boots but stalls before the menu — there is no offline mode.

This project revives the game for a single owned copy by **re-creating those backend services locally** and making the client talk to them instead of the (now unusable) real servers. The end result we are driving toward is simple and concrete: the game window reaches its real **Main Menu**, powered entirely by mock servers on this machine.

Why it is legitimate

The client is launched through its normal Easy Anti-Cheat bootstrap and runs unmodified — EAC simply starts in offline / null-client mode when its CDN is unreachable. Nothing is patched, cracked, or redistributed, and there are no other players to cheat against. It is the owner's copy, account and hardware throughout.

Definition of done

- The client, launched via the official launcher + EAC, authenticates with the owner's FXID token.
- Its backend traffic (meta, Hub, XMPP) is served by local mocks, not live servers.
- The client transitions from `InitialMenuState` to `MainMenu` and the menu is visible and interactive.

2. How the game gets online (recovered architecture)

The connection order below was reverse-engineered from a complete **known-working session log dated 12 June 2026** (captured while the real servers were still up), from packet captures, and from hundreds of recovered crash logs. Each layer must succeed for the client to reach the menu.

#	Layer	Real endpoint	Transport	Purpose
1	FXID identity	id.fx.gl / id-dev.fx.gl	HTTPS	Account login; mints the signed FXID JWT.
2	EOS + EAC	epicgames.dev, ww3.anticheat.my.games	HTTPS	Epic Online Services + anti-cheat handshake.
3	Meta / master	meta.prod.ww3.fxtools.gl:443	HTTPS	Authorises the session; returns the WW3 profile id and Hub token.
4	Hub	213.183.62.234:8705	WebSocket JSON-RPC	THE menu / lobby brain — all live menu data.
5	Presence	xmpp.prod.ww3.fxtools.gl:5222	XMPP	Friends presence and keepalive.
6	Match	per-match IP, 7867/7868	UDP	Actual in-match gameplay replication.

The authorisation handshake (the crux)

In the working log the client sends its FXID token to the **meta / master server**, which validates it and replies with the player's real WW3 profile id and a second, short-lived **Hub session token**. Only then does the client open the Hub WebSocket, carrying that token in the URL path, and drive the menu over JSON-RPC:

```
# master server authorises -> gives profile id + hub token
[Uww3PlatformManager::SetIsConnectedToMasterServer] InternalPlayerId: [1320069]

# client then opens the Hub WebSocket with a second (HS256) token in the path
ws://213.183.62.234:8705/client/<hub-session-JWT>

# menu-driving JSON-RPC over that socket
-> {"type":"RpcRequest","id":123,"context":"onlineParameters","method":"getParameters","args":[]}
<- {"type":"RpcResponse","id":123,"result":{"type":"OnlineParameters","pingPongInterval":4000,...}}
-> friends.changeStatus / notifications.getNotifications / debug.log ...
# ... immediately followed by the transition to MainMenu
```

Key implication: because the meta server is the thing that hands out the profile id and Hub token, once we control the meta response we control the whole chain — the client will accept a locally-issued identity and Hub token.

TLS is wide open

The client's libcurl runs with `bVerifyPeer = false` — it does not verify server certificates and there is no pinning. A local self-signed certificate is accepted with zero binary patching.

3. The emulation strategy

Rather than reverse-engineer and reimplement the entire backend, we implement just enough of each service to satisfy the client's startup path, and reroute the client's network traffic to those local services.

Local mock services (all on this machine)

Mock	Port	Replaces	Role
rest_server.py	443 / 80	meta.prod + public API	Friends, invitations, profile bootstrap, time; self-signed TLS.
hub_server.py	8705	Hub WebSocket	JSON-RPC menu brain: onlineParameters, notifications, friends, debug, generic fallback.
xmpp_server.py	5222	xmpp.prod	Stream, auth, bind, session, presence, keepalive.

Two ways to reroute traffic — only one works

The original approach used `/etc/hosts` entries pointing each backend hostname at `127.0.0.1`, plus a single `iptables` DNAT rule for the raw-IP Hub. Section 6 explains why the `hosts` entries never actually took effect and why **iptables DNAT is the only reliable mechanism** for this client.

The corrected principle

Redirect by destination IP at the firewall (`iptables nat OUTPUT → DNAT to 127.0.0.1`), not by hostname in `/etc/hosts`. The game's resolver bypasses `/etc/hosts` entirely.

4. What was built before this session

- A full mock backend: path-aware HTTP/HTTPS server, a WebSocket JSON-RPC Hub, and an XMPP presence server, plus an orchestration script (hosts + DNAT + teardown).
- A self-signed certificate accepted by the client's non-verifying libcurl.
- The FXID JWT schema, recovered from 264 real tokens found in crash archives: HS512, eleven required claims (sub/id, auth_type, soc_ids, email, nbf/exp/iat, iss/aud).
- A validated understanding of the menu-driving JSON-RPC protocol and the XMPP handshake, both taken from the 12 June working log.
- Proof that the official launcher, under Proton, can still log the owner in via the live FXID identity service, validate the 58 GB install, mint a real token, and start the EAC-protected game.

Where it was stuck

The mocks passed standalone tests, but no real client run had ever actually reached them. The remaining question was: what does the live client request first, and why does it never hit the mocks?

5. This session — root-cause breakthroughs

Diffing the failing runs against the 12 June working log resolved the mystery and corrected several assumptions in the original plan.

Finding 1 — the failing run used the wrong command line and a fake token

The most recent prior launch used only `-log --fxid-login-token=<synthetic>`. The token was a **synthetic** one (email `ww3-local@example.invalid`, parsed id 0.0), and three critical arguments were missing. The working June run used:

```
-pref_language english --fxid-login-token=<REAL token> -NoMrac -Region=meta.prod.ww3.fxtools.gl:443 -FXGamesInit=1
```

So the client never targeted the meta server and never contacted any backend — which is exactly why the mocks stayed silent. (The synthetic token also fails Epic's EOS Auth with HTTP 401 `external.invalid_token`, because Epic validates it against the real FXID service.)

Finding 2 — SECRETMS is a dead end

The `-Continent=SECRETMS` flag selects a different, Steam-ticket master protocol on `live.master.worldwar3.com:8443` and bypasses the entire REST/Hub/XMPP stack we can emulate. The working June session did **not** use it. Confirmed: stay on the normal FXID route.

Finding 3 — the master server, not EAC, is the gate

EOS *Connect* (needed by EAC) succeeds offline; only EOS *Auth* 401s, and that is non-fatal. The real gate to the menu is the meta/master authorisation that yields the profile id and Hub token — the piece our mocks must serve.

6. The DNS / c-ares discovery (why redirects failed)

This is the central technical breakthrough of the session and it corrects the original method.

When the correctly-armed client finally ran, it connected to the real backend IP 89.167.42.183:443 — **not** to 127.0.0.1 — even though `meta.prod.ww3.fxtools.gl` was pointed at 127.0.0.1 in `/etc/hosts`. The reason:

Root cause

The game's libcurl is built with `CURL_VERSION_ASYNCHDNS` — it resolves names with the **c-ares** asynchronous resolver, which queries DNS servers directly and **ignores `/etc/hosts`**. Every `/etc/hosts` redirect in the original plan was therefore dead weight; the client always resolved the real public IPs.

Real DNS shows what the client actually gets:

```
meta.prod.ww3.fxtools.gl -> CNAME wlg-gateway.awg.wlg.team
                        -> 89.167.40.140 , 89.167.35.180 , 89.167.42.183 (Hetzner)
xmpp.prod.ww3.fxtools.gl -> 213.183.62.234 (same host as the Hub)
```

This also explains why the Hub was the only service ever reachable in earlier tests: it was the one endpoint with an iptables **DNAT** rule (by IP), while everything else relied on the ignored hosts file.

The fix — DNAT every real backend IP to the local mocks

```
# meta / master (all three A-records) -> local HTTPS mock
iptables -t nat -A OUTPUT -d 89.167.40.140 -p tcp --dport 443 -j DNAT --to 127.0.0.1:443
iptables -t nat -A OUTPUT -d 89.167.35.180 -p tcp --dport 443 -j DNAT --to 127.0.0.1:443
iptables -t nat -A OUTPUT -d 89.167.42.183 -p tcp --dport 443 -j DNAT --to 127.0.0.1:443
# XMPP host -> local XMPP mock (Hub :8705 rule already existed)
iptables -t nat -A OUTPUT -d 213.183.62.234 -p tcp --dport 5222 -j DNAT --to 127.0.0.1:5222
sysctl -w net.ipv4.conf.all.route_localnet=1 # allow DNAT to loopback
```

Verified working

A direct probe to `https://89.167.42.183/friends/getAll` now returns the mock's `{"result": []}` — proving the real backend IP is transparently redirected to the local server end-to-end.

7. Launch mechanics & Easy Anti-Cheat

Getting the protected game to actually run took some iteration; the findings matter for repeatability.

What does and does not launch the game

Method	Result
proton run start_protected_game.exe (after a prefix teardown)	EAC bootstrap exits instantly — does not spawn the game.
proton run WW3-Win64-Shipping.exe (direct, bypassing EAC bootstrap)	Engine starts but EAC aborts it in ~60 s; no usable log.
Official launcher via Steam → click LAUNCH	Correct path: EAC bootstraps properly and the game runs with the launcher's real args.

The official launcher (steam://rungameid/674020) logs in via the live FXID service, validates files, and on **LAUNCH** starts the game through EAC with the correct region arguments. In the last run this reached a live GameThread that dialed the real meta IP — the exact behaviour the DNAT fix now captures.

Lesson

Do not tear the Proton prefix down with wineserver -k between attempts — it breaks the EAC bootstrap. Launch through the official launcher and let it drive EAC.

8. Current system state

Component	State
HTTPS mock (meta)	Up on 0.0.0.0:443, live-logging, verified via probe.
HTTP mock	Up on 0.0.0.0:80.
Hub mock	Up on 0.0.0.0:8705.
XMPP mock	Up on 127.0.0.1:5222.
DNAT: meta IPs	89.167.40.140 / .35.180 / .42.183 :443 → 127.0.0.1:443 (NEW , verified).
DNAT: Hub	213.183.62.234:8705 → 127.0.0.1:8705.
DNAT: XMPP	213.183.62.234:5222 → 127.0.0.1:5222 (NEW).
route_localnet	Enabled (DNAT-to-loopback works).
Official launcher	Running, signed in as GEORGEKGAMING[STEAM], "Ready to Launch".
Live FXID token	Present locally, mode 600, ~7-day validity. Value never exposed.

Immediate next action

Everything backend-side is armed and verified for the first time. The single remaining step is to press **LAUNCH** so the game runs with the real backend IPs now routed to the mocks — then observe the first meta → Hub → XMPP requests and fill in any exact response shapes still needed.

9. Remaining work to reach Main Menu

- Launch the game via the official launcher and confirm the meta request now lands on the HTTPS mock (watch the live logs in /tmp/ww3mock).
- Serve a correct meta / master authorisation response: a WW3 profile id plus a Hub session token, so the client fires `SetIsConnectedToMasterServer` and opens the Hub WebSocket.
- Answer the Hub JSON-RPC opening calls (`onlineParameters.getParameters`, `notifications.getNotifications`, `friends.changeStatus`, `debug.log`) with the exact shapes from the June working log, keeping request ids correlated.
- Complete the XMPP stream/auth/bind/session/presence handshake for the mapped player id.
- Confirm the client log transitions `InitialMenuState` → `MainMenu` and the menu is visible.
- Then run the teardown (remove DNAT rules, stop mocks) to restore the machine to a clean network state.

Expected end state: a visible, interactive Main Menu driven entirely by local mock services, with logs showing the meta bootstrap, Hub RPC exchange and XMPP session against 127.0.0.1 — the discontinued game running again on the owner's own machine.

10. Glossary

Term	Meaning
FXID	The game's identity service (id.fx.gl); issues the signed JWT the client logs in with.
Meta / master server	Backend that authorises the session and returns the WW3 profile id + Hub token.
Hub	WebSocket JSON-RPC service that drives the menu and lobby (port 8705).
EOS	Epic Online Services SDK bundled with the game (Connect + Auth interfaces).
EAC	Easy Anti-Cheat; here it runs normally in offline / null-client mode.
c-ares	Async DNS resolver compiled into the game's libcurl; ignores /etc/hosts.
DNAT	iptables destination rewriting — redirects a real backend IP to 127.0.0.1.
SECRETMS	An alternate Steam-ticket master protocol (port 8443); deliberately NOT used.
Proton	Valve's Wine-based compatibility layer that runs the Windows client on Linux.